

Revisão de POO — Sistema de Pedidos

Atividade de Revisão Resolvida e Comentada · Java · 2º Semestre ADS
Encapsulamento · Herança · Polimorfismo · Interfaces · Composição · Arquivos

Como usar este material: cada questão traz o código completo seguido da explicação do conceito cobrado. As respostas discursivas ("explique com suas palavras") estão escritas de forma direta, do jeito que você pode reproduzir na prova. No final há a Questão 8 (teoria) respondida e uma lista de erros clássicos que derrubam nota.

Questão 1 — Classe Pessoa (Encapsulamento)

Atributos **protected**, construtor completo, getters/setters e o método `mostrar()`.

```
public class Pessoa {
    protected String cpf;
    protected String nome;
    protected String email;
    protected String senha;

    public Pessoa(String cpf, String nome, String email, String senha) {
        this.cpf = cpf;
        this.nome = nome;
        this.email = email;
        this.senha = senha;
    }

    public String getCpf() { return cpf; }
    public void setCpf(String cpf) { this.cpf = cpf; }

    public String getNome() { return nome; }
    public void setNome(String nome) { this.nome = nome; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    public String getSenha() { return senha; }
    public void setSenha(String senha) { this.senha = senha; }

    public void mostrar() {
        System.out.println(nome);
    }
}
```

Explicação

Encapsulamento é esconder o estado interno do objeto e controlar o acesso a ele por meio de métodos (getters/setters). Aqui o enunciado pediu `protected` em vez de `private`: a diferença é que `protected` permite acesso direto pelas **subclasses** (e pelo mesmo pacote), enquanto `private` restringe à própria classe. Isso é útil porque `Usuario` (Questão 3) vai herdar de `Pessoa` e poderá usar `nome`, `cpf` etc. diretamente, sem precisar de `getter`.

■ **Pegadinha de prova:** `protected` ainda é encapsulamento — só é menos restritivo que `private`. Se cair "protected quebra o encapsulamento?", a resposta é não: ele *flexibiliza* o acesso para a hierarquia de herança.

Questão 2 — Classe Produto (Construtor Padrão)

Versão inicial (antes da Questão 6 transformá-la em abstrata):

```

public class Produto {
    private int id;
    private String desc;
    private Double preco;

    // Construtor padrao (sem parametros)
    public Produto() {
        this.id = 0;
        this.desc = "";
        this.preco = 0.0;
    }

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getDesc() { return desc; }
    public void setDesc(String desc) { this.desc = desc; }

    public Double getPreco() { return preco; }
    public void setPreco(Double preco) { this.preco = preco; }

    public void mostrar() {
        System.out.println(desc);
    }
}

```

Explicação

Construtor padrão é o construtor sem parâmetros. Detalhe importante: se você não escrever *nenhum* construtor, o Java cria um implícito; mas se você declarar qualquer construtor com parâmetros, o padrão deixa de existir automaticamente — por isso a questão pede para escrevê-lo explicitamente. Adicionei getters/setters porque a classe Pedido (Q5) precisa ler preco e id de fora.

Note também que preco é Double (classe wrapper) e não double (primitivo). Wrapper aceita null e tem métodos utilitários; é o tipo pedido no enunciado.

Questão 3 — Classe Usuario (Herança)

```

public class Usuario extends Pessoa {
    private int anoNascimento;

    public Usuario(String cpf, String nome, String email,
        String senha, int anoNascimento) {
        super(cpf, nome, email, senha); // reaproveita o construtor de Pessoa
        this.anoNascimento = anoNascimento;
    }

    public int getAnoNascimento() { return anoNascimento; }
    public void setAnoNascimento(int anoNascimento) {
        this.anoNascimento = anoNascimento;
    }
}

```

Explicação

(b) Por que Usuario não precisa redeclarar cpf, nome, email e senha? Porque a herança (extends Pessoa) faz com que Usuario **já possua** todos os atributos e métodos não-privados da superclasse. Redefinir seria duplicação de código — exatamente o que a herança existe para evitar. Como os atributos são protected, Usuario pode inclusive acessá-los diretamente.

■ **Regra do super():** a chamada super(. . .) deve ser a **primeira linha** do construtor da subclasse. Se Pessoa não tem construtor vazio e você esquecer o super(), o código nem compila.

Questão 4 — Interface Registravel

```
public interface Registravel {  
    void registrar();           // sem implementacao (metodo abstrato)  
    String gerarResumo();       // retorna texto representando o objeto  
}
```

Explicação

Uma **interface** é um contrato puro: declara *o que* a classe deve fazer, sem dizer *como*. Em interfaces, os métodos são implicitamente `public abstract`, então não precisa escrever esses modificadores.

(c) Interface × classe abstrata — resposta pronta para a prova:

- **Classe abstrata** pode ter atributos de instância, construtores e métodos concretos (com corpo) além dos abstratos. Uma classe só pode estender **uma** classe abstrata (herança simples).
- **Interface** define apenas o contrato (assinaturas). Não tem construtor nem estado de instância. Uma classe pode implementar **várias** interfaces.
- **Quando usar cada uma:** use classe abstrata quando existe relação "é um" com código/atributos em comum para reaproveitar (ex.: Produto é a base de ProdutoFisico e ProdutoDigital, compartilhando id, desc e preco). Use interface quando classes *sem parentesco* precisam garantir a mesma capacidade (ex.: qualquer entidade "registrável" em arquivo — Pedido hoje, talvez Usuario amanhã).

Questão 5 — Classe Pedido (Composição + ArrayList + Interface)

```
import java.io.BufferedWriter;  
import java.io.FileWriter;  
import java.io.IOException;  
import java.util.ArrayList;  
  
public class Pedido implements Registravel {  
    private int id;  
    private Usuario usuario;  
    private ArrayList<Produto> produtos;  
    private Double valorPedido;  
  
    // a) construtor: recebe Usuario, id e valor inicial; lista vazia  
    public Pedido(Usuario usuario, int id, Double valorInicial) {  
        this.usuario = usuario;  
        this.id = id;  
        this.valorPedido = valorInicial;  
        this.produtos = new ArrayList<>();  
    }  
  
    // b) adiciona produto e soma o preco  
    public void adicionarProduto(Produto p) {  
        produtos.add(p);  
        valorPedido += p.getPreco();  
    }  
  
    // c) remove produto e subtrai o preco  
    public void removerProduto(Produto p) {  
        if (produtos.remove(p)) {  
            valorPedido -= p.getPreco();  
        }  
    }  
  
    // d) percorre a lista chamando mostrar() em cada produto  
    public void listarProdutos() {  
        for (Produto p : produtos) {  
            p.mostrar(); // chamada polimorfica!  
        }  
    }  
}
```

```

// ... continuacao da classe Pedido ...

// e) busca produto pelo id; null se nao encontrar
public Produto consultarProduto(int id) {
    for (Produto p : produtos) {
        if (p.getId() == id) {
            return p;
        }
    }
    return null;
}

// f) SOBRECARGA: mesmo nome, parametros diferentes
public void removerProduto(int id) {
    Produto p = consultarProduto(id);
    if (p != null) {
        removerProduto(p); // reutiliza a versao que recebe Produto
    }
}

// g) grava em pedido.txt (modo append) com try/catch
@Override
public void registrar() {
    try (BufferedWriter bw =
        new BufferedWriter(new FileWriter("pedido.txt", true))) {
        for (Produto p : produtos) {
            bw.write(id + ";" + usuario.getCpf() + ";"
                + p.getId() + ";" + p.getPreco());
            bw.newLine();
        }
    } catch (IOException e) {
        System.out.println("Erro ao registrar pedido: " + e.getMessage());
    }
}

// h) resumo do pedido
@Override
public String gerarResumo() {
    return "Pedido #" + id + " | Cliente: " + usuario.getNome()
        + " | Total: R$ " + valorPedido;
}
}

```

Explicação

Composição: Pedido **tem um** Usuario e **tem uma** lista de Produtos. Repare a diferença para a herança: Usuario é *uma* Pessoa (extends), enquanto Pedido é *composto por* objetos de outras classes guardados como atributos.

(f) Conceito aplicado: sobrecarga (overloading) — dois métodos com o mesmo nome e **assinaturas diferentes** (um recebe Produto, outro recebe int). O compilador escolhe qual chamar pelos argumentos, em **tempo de compilação**.

(g) Arquivo: o `new FileWriter("pedido.txt", true)` com o segundo argumento `true` é o que ativa o **modo append** (acrescentar no fim em vez de sobrescrever). O `BufferedWriter` envolve o `FileWriter` para escrever com buffer (mais eficiente). Usei **try-with-resources** (o recurso declarado dentro dos parênteses do `try` é fechado automaticamente) — se o professor ainda não mostrou essa forma, a alternativa é chamar `bw.close()` dentro do `try` ou em um `finally`.

■ **Erros que derrubam nota aqui:** esquecer os imports de `java.io` e `java.util`; esquecer o `true` do `append`; e esquecer de atualizar `valorPedido` ao adicionar/remover.

Questão 6 — Polimorfismo (Produto abstrata + subclasses)

(a) Produto vira classe abstrata e mostrar() vira abstrato. Atenção: a partir daqui new Produto() deixa de compilar — classe abstrata não pode ser instanciada. O construtor continua existindo, mas só é chamado via super() pelas filhas.

```
public abstract class Produto {
    protected int id;
    protected String desc;
    protected Double preco;

    public Produto() {
        this.id = 0;
        this.desc = "";
        this.preco = 0.0;
    }

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getDesc() { return desc; }
    public void setDesc(String desc) { this.desc = desc; }
    public Double getPreco() { return preco; }
    public void setPreco(Double preco) { this.preco = preco; }

    public abstract void mostrar(); // sem corpo: as filhas SAO OBRIGADAS a implementar
}
```

```
public class ProdutoFisico extends Produto {
    private Double pesoKg;

    public ProdutoFisico(int id, String desc, Double preco, Double pesoKg) {
        super(); // inicializa com os valores padrao
        this.id = id; // protected: acesso direto permitido
        this.desc = desc;
        this.preco = preco;
        this.pesoKg = pesoKg;
    }

    public Double getPesoKg() { return pesoKg; }
    public void setPesoKg(Double pesoKg) { this.pesoKg = pesoKg; }

    @Override
    public void mostrar() {
        System.out.println("[FISICO] " + desc + " - R$ " + preco
            + " (peso: " + pesoKg + " kg)");
    }
}
```

```
public class ProdutoDigital extends Produto {
    private String linkDownload;

    public ProdutoDigital(int id, String desc, Double preco, String linkDownload) {
        super();
        this.id = id;
        this.desc = desc;
        this.preco = preco;
        this.linkDownload = linkDownload;
    }

    public String getLinkDownload() { return linkDownload; }
    public void setLinkDownload(String linkDownload) { this.linkDownload = linkDownload; }

    @Override
    public void mostrar() {
        System.out.println("[DIGITAL] " + desc + " - R$ " + preco
            + " (download: " + linkDownload + ")");
    }
}
```

Explicação

(d) O que se observa na saída? Cada elemento da lista imprime de um jeito diferente: o físico mostra peso, o digital mostra link — mesmo todos sendo tratados como Produto. Isso demonstra **polimorfismo**: a JVM decide **em tempo de execução** qual versão de mostrar() chamar, com base no tipo real do objeto (dynamic dispatch).

(e) Por que dá pra guardar ProdutoFísico e ProdutoDigital num ArrayList<Produto>? Porque toda subclasse **é um** Produto (relação de herança). Um objeto ProdutoFísico pode ser referenciado por uma variável do tipo da superclasse — isso se chama **upcasting** e é automático. A lista declara o tipo da referência (Produto), mas cada posição guarda um objeto cujo tipo real é da subclasse.

■ **Pegadinha clássica:** ao sobrescrever, a assinatura e o tipo de retorno devem bater com o método da superclasse. Usar `@Override` faz o compilador avisar se você errar o nome ou os parâmetros — sempre use.

Questão 7 — Classe App (Integração)

```
import java.util.ArrayList;

public class App {
    public static void main(String[] args) {
        // a) usuario
        Usuario u = new Usuario("123.456.789-00", "Matheus",
                                "matheus@email.com", "senha123", 1990);

        // b) produtos (um fisico e um digital)
        ProdutoFisico livro = new ProdutoFisico(1, "Livro Java", 89.90, 0.6);
        ProdutoDigital ebook = new ProdutoDigital(2, "E-book P00", 29.90,
                                                  "https://loja.com/ebook");

        // c) pedido + produtos + registrar()
        Pedido pedido = new Pedido(u, 1, 0.0);
        pedido.adicionarProduto(livro);
        pedido.adicionarProduto(ebook);
        pedido.registrar(); // grava em pedido.txt

        // d) resumo
        System.out.println(pedido.gerarResumo());

        // e) saida polimorfica
        pedido.listarProdutos();

        // Demonstracao extra da Q6-d:
        ArrayList<Produto> lista = new ArrayList<>();
        lista.add(livro);
        lista.add(ebook);
        for (Produto p : lista) {
            p.mostrar(); // cada um responde do seu jeito
        }
    }
}
```

Saída esperada no console:

```
Pedido #1 | Cliente: Matheus | Total: R$ 119.8
[FISICO] Livro Java - R$ 89.9 (peso: 0.6 kg)
[DIGITAL] E-book P00 - R$ 29.9 (download: https://loja.com/ebook)
[FISICO] Livro Java - R$ 89.9 (peso: 0.6 kg)
[DIGITAL] E-book P00 - R$ 29.9 (download: https://loja.com/ebook)
```

E o arquivo `pedido.txt` ganha as linhas:

```
1;123.456.789-00;1;89.9
1;123.456.789-00;2;29.9
```

Questão 8 — Reflexão Teórica (respostas prontas)

a) Qual pilar é representado pelo `protected` em `Pessoa`?

O **encapsulamento**. Os atributos não são públicos: o acesso externo é feito por getters/setters, que controlam como o estado do objeto é lido e alterado. O `protected` apenas flexibiliza esse controle para as subclasses (`Usuario` acessa os atributos herdados diretamente), mantendo-os protegidos do restante do código.

b) Diferença entre herança e composição + exemplos no sistema.

Herança é a relação "**é um**": a subclasse recebe atributos e comportamento da superclasse. Exemplos: `Usuario` `extends` `Pessoa` e `ProdutoFisico` `extends` `Produto`.

Composição é a relação "**tem um**": uma classe é construída usando objetos de outras como atributos. Exemplo: `Pedido` tem um `Usuario` e um `ArrayList<Produto>`.

c) `removeProduto` com duas assinaturas — como se chama e para que serve?

Sobrecarga de métodos (overloading). Utilidade: oferecer formas diferentes e convenientes de realizar a mesma operação (remover pelo objeto ou pelo id), melhorando a usabilidade da classe sem inventar nomes diferentes para a mesma ação.

d) Sobrecarga × sobrescrita.

Sobrecarga (overloading): métodos com o **mesmo nome** e **parâmetros diferentes** na **mesma classe**; resolvida em **tempo de compilação**. Exemplo: as duas versões de `removeProduto` em `Pedido`.

Sobrescrita (overriding): a **subclasse redefine** um método herdado com a **mesma assinatura**; resolvida em **tempo de execução** (dynamic dispatch). Exemplo: `ProdutoFisico` e `ProdutoDigital` sobrescrevendo `mostrar()` de `Produto`.

e) `Registravel r = new Pedido(...)` — o que acontece e quais métodos ficam acessíveis?

É perfeitamente válido (upcasting para o tipo da interface). Pela referência `r`, só ficam acessíveis os métodos **declarados na interface**: `registrar()` e `gerarResumo()`. Métodos próprios de `Pedido`, como `adicionarProduto()`, não são visíveis por essa referência — seria preciso fazer um cast de volta para `Pedido`. Importante: ao chamar `r.registrar()`, executa a implementação de **`Pedido`**, porque o tipo real do objeto é quem manda em tempo de execução (polimorfismo).

Checklist final — erros que derrubam nota na prova

1. **Esquecer imports**: `java.util.ArrayList`, `java.io.FileWriter`, `java.io.BufferedWriter`, `java.io.IOException`.
2. **`super()` fora da primeira linha** do construtor da subclasse (ou ausente quando o pai não tem construtor vazio).
3. **Instanciar classe abstrata**: depois da Q6, `new Produto()` não compila.
4. **Esquecer o `true` do `FileWriter`** — sem ele o arquivo é sobrescrito em vez de receber `append`.
5. **Não implementar todos os métodos da interface** — o compilador acusa erro em `Pedido` se faltar `registrar()` ou `gerarResumo()`.
6. **Confundir sobrecarga com sobrescrita** — sobrecarga = mesma classe, parâmetros diferentes; sobrescrita = subclasse, mesma assinatura.
7. **Trocar o tipo de retorno ao sobrescrever** — a assinatura deve bater com a da superclasse (use `@Override` para o compilador conferir).
8. **Não atualizar `valorPedido`** ao adicionar/remover produtos.